Prompting as Specification, Not Magic Words

FRIDAY AI Education — Episode 016 · 2026-07-02 · Printable Companion

FRIDAY AI Education — Episode 016 · Printable Companion Paper

Abstract

Most people treat a prompt as an incantation: they phrase and rephrase until the model produces something acceptable, then move on without knowing *why* it worked. This is a fragile way to build anything. A more durable stance is to treat a prompt as a **specification** — the same kind of document you would hand a competent contractor. A good spec states the goal, supplies the inputs, fixes the constraints, defines the shape of the output, sets a quality bar, and says what to do when things go wrong. This paper develops that stance from first principles, shows the mechanism underneath it, works through three concrete prompts an operator actually writes (a brief, a research task, a workflow diagnostic), and ends by connecting the idea to durable systems — memory, retrieval, and human approval gates — rather than one-off chats. The claim is narrow and, once seen, hard to unsee: **the quality of what you get out of a language model is bounded by the quality of the specification you put in.**

1. Why this matters before you write a line of it

You are building a company on top of these systems, not just using them to draft emails. That changes the stakes. If a prompt is a clever phrase you found by trial and error, it lives in your head, it cannot be reviewed, it cannot be improved by anyone else, and it breaks silently when the model updates or the inputs shift. If a prompt is a specification, it is an *asset*: it can be read, criticized, versioned, tested, and handed to someone else. The difference between those two worlds is the difference between a personal trick and an operating system.

There is a quieter reason too. Being taken seriously in a technical conversation rarely requires you to write the model's internals on a whiteboard. It requires you to reason clearly about *what you are asking a system to do and how you would know if it did it.* People who think a prompt is magic words say things like "it just needs better wording." People who think a prompt is a spec ask, "what output shape did you require, and what was your acceptance test?" The second question is the one that ends the argument. This paper is training for asking the second question.

We are going to stay narrow on purpose. There is a large surrounding world — retrieval, fine-tuning, tool use, evaluation harnesses — and we will touch the edges of it only where it clarifies the core. The core is one idea: **specification.**

2. The core idea in plain language

A specification is a description of a desired result precise enough that a competent party can produce that result *and* precise enough that you can check whether they did. Hold both halves in mind. Precision that produces but cannot be checked is a wish. Precision that checks but cannot produce is a complaint.

A prompt, written well, is exactly this document aimed at a language model instead of a person. It has six parts, and it is worth learning them as a fixed checklist because the failures of bad prompts map almost perfectly onto missing

parts:

1. **Goal** — what result you actually want, stated as an outcome, not an activity. "Summarize this" is an activity. "Produce a summary a busy investor could read in ninety seconds and correctly decide whether to take the meeting" is an outcome.
2. **Inputs** — the specific material the model should work *from*, clearly separated from your instructions. If the model has to guess what the source material is, it will hallucinate the boundary.
3. **Constraints** — what must be true of the result and what is forbidden. Length, tone, what not to invent, what to exclude, what authority to defer to.
4. **Output shape (schema)** — the structure the answer must take. Prose? A table with named columns? JSON with required fields? This is the single most under-used lever operators have.
5. **Quality bar (acceptance criteria)** — the test you will apply to decide whether the output is good enough. If you cannot name the test, you have not finished specifying.
6. **Failure handling** — what the model should do when it cannot meet the spec: when the input is missing, ambiguous, or contradictory. The default failure mode of an unspecified model is *confident guessing*, which is the worst option in an operating context.

The rest of this paper is these six parts, examined, and then applied.

> **The reframe.** The unit of skill is not *phrasing*. It is *specification*. A person who cannot phrase elegantly but specifies completely will get better, more reliable output than a person who phrases beautifully and specifies nothing.

3. The mechanism underneath — enough, and no more

You do not need the mathematics, but you do need an accurate mental model, because the wrong model produces the wrong prompts.

A large language model is a system that, given a sequence of text, produces a probability distribution over what text plausibly comes next, and then extends the sequence accordingly. It was trained on an enormous body of human text, so the patterns it has absorbed are patterns of *how text tends to continue* — including the patterns of well-structured documents, arguments, tables, and code. It is a generation engine operating over learned regularities in language. That is the whole of it, and it is a great deal.

Three consequences follow, and each one justifies a part of the specification checklist.

First: the model continues your prompt; it does not read your mind. Everything it will condition on has to be present in the text you provide (plus whatever tools or retrieved material you attach). If a constraint matters and you did not write it, the model is not withholding it — it never had it. This is why *inputs* and *constraints* are not optional politeness; they are the entire basis of the response. A person can infer "obviously you don't want me to make up the revenue figures." A model has no privileged access to "obviously." Say it.

Second: the model is fluent regardless of whether it is correct. Fluency and accuracy are produced by the same machinery — plausible continuation — so a confident, well-formed wrong answer costs the model no more effort than a right one. This is not a bug you can phrase your way out of; it is a property of the mechanism. It is precisely *why* the specification must include an acceptance test and a failure instruction. You are not asking the model to be humble by nature. You are building the check outside the model. (And note: the model does not "think like a person." It has no beliefs it is choosing to share or hide. Treating it as a colleague with hidden knowledge is the source of a whole family

of prompting mistakes.)

Third: structure in the prompt propagates to structure in the output. Because the model continues patterns, a prompt that *is* well-organized — clear sections, named fields, an explicit example — pulls the output toward being well-organized in the same way. This is the leverage behind output schemas and examples, discussed below. You are, in a real sense, showing the model the shape of the answer and letting its pattern-completion do the rest.

None of this requires the model to be intelligent in the human sense for the specification stance to pay off. It only requires the model to be a powerful, structure-sensitive continuation engine — which it is. The spec is how you aim it.

4. Output schema: the highest-leverage part

If you change one habit after this paper, change this one: **decide the shape of the answer before you ask the question.**

An output schema is a description of the *structure* the answer must have, independent of its content. "Three paragraphs, no headers." "A markdown table with columns: Claim, Source, Confidence, Caveat." "A JSON object with fields `decision` (one of: proceed, hold, reject), `reasons` (array of strings), `missing_info` (array of strings)." The schema does two things at once, and both are valuable.

It **constrains the model** toward the form you can actually use. A model asked for "thoughts on this vendor" will return an essay. The same model asked for "a table scoring this vendor on price, integration effort, lock-in risk, and support quality, one row per dimension, with a one-line justification each" returns something you can drop into a decision. Same underlying knowledge, radically different usability.

It **makes the output checkable.** A named structure is a structure you can inspect programmatically or at a glance. If you required a `missing_info` field and it comes back empty on a question you know was under-specified, that empty field is a signal. Free-form prose hides that signal inside good grammar.

There is a subtler benefit. When you force yourself to design the schema, you often discover that *you* did not know what you wanted. The discipline of naming the columns is the discipline of deciding what the decision actually depends on. The schema is a thinking tool before it is a formatting tool.

A caution, so this does not become cargo cult: schema is a means, not a virtue. A rigid JSON structure imposed on a task that genuinely wants a paragraph of nuance will strip out the nuance. Match the shape to the decision the output feeds. The question is always "what form makes this *usable and checkable*," not "how structured can I make this look."

5. Acceptance criteria: the test you name in advance

Acceptance criteria are the conditions the output must satisfy to count as done — written *before* you see the output, so that "good enough" is a standard and not a mood. In software this is routine; a feature is not complete because it looks finished but because it passes a stated test. Prompts deserve the same rigor and rarely get it.

Concretely, acceptance criteria answer: *how will I know this is good without re-doing the work myself?* Examples:

- "Every factual claim is attributable to a provided source; no claim rests on the model's general knowledge."

- "The brief fits on one page and a non-expert can act on it without a follow-up question."
- "The table has exactly one row per option and no cell says 'it depends' without saying on what."
- "If the model is under 70% confident on any line, that line is flagged, not smoothed over."

Two things make acceptance criteria powerful. They are the bridge between a prompt and a **test**: once you can state the criteria, you can check outputs against them at scale, which is the beginning of evaluations — the systematic testing of whether a prompt or system meets its bar across many cases, not just the one you happened to look at. And they *discipline the goal*. If you cannot write the acceptance test, your goal is still vague, and no amount of clever phrasing will rescue it. The inability to name the test is diagnostic: it means you have not finished thinking.

Write the acceptance criteria into the prompt where you can — telling the model the bar it will be judged against measurably improves the output — but keep a copy for yourself, because *you* are the final gate. The model can be told the standard; it cannot be trusted to be the one who enforces it.

6. Failure modes: specify what happens when it can't comply

The most dangerous property of an unspecified prompt is what the model does when it *cannot* do what you asked. Left unspecified, it does the worst possible thing: it produces a fluent, confident answer anyway. Missing data becomes an invented figure. An ambiguous request becomes a confidently wrong interpretation. A contradiction in the inputs becomes silently resolved in whatever direction the text happened to flow.

The fix is to make failure a *specified, first-class output* rather than an accident. You do this by telling the model, explicitly, what to do when the spec cannot be met:

- "If a required figure is not present in the provided material, write `NOT PROVIDED` for that field. Do not estimate."
- "If the request is ambiguous in a way that changes the answer, stop and list the clarifying questions instead of guessing."
- "If two sources conflict, surface the conflict; do not pick a winner."
- "If you are extrapolating beyond the provided inputs, mark that sentence as an inference."

This single move — giving the model a legitimate way to say "I can't, and here's why" — converts a large class of silent, confident errors into visible, actionable signals. In an operating context that is the whole game. A visible gap is a task; a hidden fabrication is a landmine. Note that this is a spec-level fix, not a phrasing trick: you are not coaxing honesty out of the model's character, you are defining an output it is allowed to produce and rewarding it for producing that output over a fabricated one.

The table below is the checklist in one view — the failure the model tends toward, and the part of the spec that prevents it.

Spec part	The question it answers	Failure it prevents
Goal	What outcome do I actually want?	Busywork: fluent output that solves the wrong problem
Inputs	What material should it work from?	Hallucinated context; invented sources
Constraints	What must be true / is forbidden?	Off-tone, over-long, or out-of-bounds output
Output schema	What shape must the answer take?	Unusable prose you must re-process by hand

Spec part	The question it answers	Failure it prevents
Acceptance criteria	How will I know it's good enough?	"Good enough" as a mood; silent quality drift
Failure handling	What to do when it can't comply?	Confident guessing in place of an honest gap

7. Examples: showing beats describing

There is a seventh element worth its own section because it is so often the difference between a mediocre and an excellent prompt: a worked **example**. Because the model completes patterns, one concrete instance of the input-and-desired-output pair teaches it more precisely than a paragraph of adjectives. "Write in a crisp, executive tone" is an instruction the model must interpret. A three-line sample of exactly the crisp executive tone you mean is a pattern it can match. Show, don't only tell.

The OpenAI prompt engineering guide (linked in the reading section) makes this one of its central tactics, and it is worth internalizing the reason rather than just the rule: examples work not because the model is "learning" in the human sense during your session, but because they narrow the space of plausible continuations to the region you actually want. One good example is often worth more than three sentences of description — and, importantly, a *bad* or off-target example is worse than none, because it aims the pattern-completion at the wrong target. Choose examples the way you would choose a reference sample for a contractor: representative, correct, and in the exact register you want back.

8. Three prompts an operator actually writes

Theory earns its keep in application. Here are three prompts of the kind you write while building a company, each shown first as most people write it and then as a specification. Read them as templates, not scripts.

8.1 The brief prompt

You want a one-page brief on a potential partner before a call.

As most people write it:

> "Give me a brief on Acme Corp before my meeting."

The model will produce a fluent, generic summary, likely padded with general knowledge that may be outdated or invented, in whatever length and shape it chooses. You will read it, distrust half of it, and do the work yourself anyway.

As a specification:

> **Goal:** Produce a one-page pre-meeting brief that lets me walk into a call with Acme's Head of Partnerships and sound informed, and decide whether a pilot is worth proposing.

>

> **Inputs:** Use only the three documents pasted below — Acme's public product page, their last press release, and the email thread with their team. [documents follow, clearly delimited]

>

> **Constraints:** No claims drawn from your general knowledge of Acme; everything must trace to the provided documents. Neutral, factual tone. Under 400 words.

>

> **Output schema:** Four sections — (1) What they do, three sentences. (2) Why they're talking to us, bulleted. (3) Two questions I should ask, with the reasoning. (4) One risk or open question about the partnership.

>

> **Acceptance criteria:** Every factual sentence is attributable to one of the three documents. A colleague who never saw the documents could run the meeting from this brief alone.

>

> **Failure handling:** If a section cannot be supported by the provided documents, write "Not established in provided materials" rather than filling it in.

The second version is longer to write once and produces something you can trust and reuse. Notice that most of the added length is not cleverness — it is *removing the model's permission to guess.*

8.2 The research prompt

You want to understand a market before committing a week to it. Research is where confident fabrication does the most damage, so the failure handling carries most of the weight.

As a specification:

> **Goal:** Give me a structured, source-grounded view of how mid-market companies currently make [X] decisions, good enough to decide whether there's a wedge worth exploring.

>

> **Inputs:** Work from the sources I provide / retrieve; where you use general knowledge, mark it as such and treat it as a lead to verify, not a fact.

>

> **Constraints:** Distinguish clearly between what a source states and what you are inferring. No market-size numbers unless a source is cited; a plausible-sounding figure with no source is worse than "unknown."

>

> **Output schema:** A table with columns: Finding · Source (or "inference") · Confidence (high/med/low) · What would confirm or kill it.

>

> **Acceptance criteria:** I can act on every "high confidence" row without re-checking it. Every "low confidence" row names the specific thing that would resolve it.

>

> **Failure handling:** Where the provided sources are silent, say so and add the finding to a short "Open questions" list rather than reasoning your way to an answer.

The value here is not that the model does your research. It is that the output separates *knowledge* from *guess* structurally, so your scarce verification effort goes exactly where the uncertainty is. That "What would confirm or kill it" column turns a research dump into a research *plan*.

8.3 The workflow diagnostic prompt

This is the operator's power tool. You have a process that is misbehaving — an onboarding flow, a support queue, an internal handoff — and you want the model to analyze it rather than perform it.

As a specification:

> **Goal:** Diagnose why our new-customer onboarding takes eleven days when we target three, and identify the highest-leverage intervention.
>
> **Inputs:** The step-by-step process description and the timestamps from our last ten onboardings, pasted below.
>
> **Constraints:** Diagnose only from the provided data. Do not propose tooling we haven't mentioned; work within the current process. Separate "where time is lost" from "why."
>
> **Output schema:** (1) A ranked list of the three steps consuming the most elapsed time, with the numbers. (2) For the top step, the two most likely causes given the data. (3) One change to test first, and what metric would show it worked.
>
> **Acceptance criteria:** Each cause is grounded in a pattern actually visible in the timestamps, not a generic "handoffs are slow." The proposed test is something we could run next week.
>
> **Failure handling:** If the data is insufficient to locate the bottleneck, say what specific measurement is missing rather than offering a generic best-practices list.

The generic-best-practices list is the model's default failure mode for diagnostic questions, and it is seductive because it sounds wise. The failure-handling clause explicitly forecloses it: *if you can't find it in the data, tell me what data you'd need.* That is the difference between a consultant's brochure and an actual diagnosis.

9. Connecting it to durable systems

Everything so far concerns a single prompt. The reason this matters for company-building is that a company does not run on single prompts — it runs on *systems*, and a specification is the component such systems are built from.

FRIDAY is meant to be an orchestrator: something that carries context, holds memory, uses tools, and produces reliable work across many steps — not a chat window you retype into. That ambition rests directly on the specification stance, for a simple reason. **A well-specified prompt is a reusable component; a lucky phrasing is not.** You can store a spec, version it, test it against many cases, and let a system call it a thousand times. You cannot do any of that with a phrase you stumbled onto and cannot explain.

Two connections are worth drawing precisely, because they are where the individual skill becomes organizational capability.

Memory and retrieval as inputs. A knowledge substrate — call it GBrain in this project's terms — exists so that the right *inputs* are available to the model at the moment it works, without a human pasting them in. The common technique here is **retrieval-augmented generation (RAG)**: before the model answers, a retrieval step pulls the most relevant material from a store and adds it to the prompt as inputs. That retrieval typically works by **semantic similarity** — comparing **vector distance** between a representation of your query and representations of stored documents, so that "relevant" means "close in meaning," not close in wording. Two cautions belong here and matter for how seriously you take the output. Those vector representations (embeddings) capture *similarity*, not *truth*; a document can be highly similar to your query and completely wrong, so retrieval decides what the model *sees*, not what is *correct* — the acceptance criteria and failure handling still do the truth-work. This is why the "cite your sources / mark inferences" clauses from Section 8 are not optional decoration in a retrieval system; they are the load-bearing wall. The spec stance and the memory substrate are complements: memory supplies the inputs, the spec governs what is done with them and how the result is checked.

Governance as specification at the system level. Useful organizational AI needs more than a good prompt: it needs permissions (what the system is allowed to touch), evaluations (systematic tests that the outputs still meet the bar as models and inputs change), audit (a record of what was done and on what basis), and human approval gates (a person in the loop before consequential actions execute). Read that list again and notice that it is the same six-part checklist, promoted from one prompt to a whole workflow. Constraints become permissions. Acceptance criteria become evaluations. Failure handling becomes approval gates. An "agent" is not a tireless employee who exercises judgment — it is a specified workflow of model calls, tools, and checks, and it is exactly as trustworthy as its specification and its gates make it. Automation is safe to the degree that this scaffolding exists, and reckless to the degree it doesn't. The person who writes prompts as specs has already learned, at small scale, the exact discipline that governs systems at large scale.

For the first practical wedge — making a person's or company's public body of work searchable, citable, and operationally useful — every one of these threads converges. The corpus is the input; retrieval selects it by semantic similarity; the spec demands citation and marks inference; acceptance criteria enforce that every answer traces to the source; failure handling ensures "the corpus doesn't say" is a valid, visible answer rather than a fabricated one. That product *is* the specification stance, instantiated.

10. The main limitation, stated honestly

A specification does not make the model correct. It makes the model's output *shaped, checkable, and honest about its gaps* — which is a large improvement, and not the same thing as correct.

The model can satisfy every structural requirement of your schema and still be wrong inside a cell. It can cite a provided source and still misread it. It can follow your failure-handling instruction most of the time and ignore it occasionally, because it is a probabilistic system, not a rule-follower. Specification narrows the space of outputs toward what you want and surfaces the model's uncertainty where you told it to; it does not convert a plausible-continuation engine into an oracle.

This is why the specification stance and the *verification* stance are two halves of one competence. The spec gets you an output you can check efficiently. You — or an evaluation system, or an approval gate — still have to do the checking on anything that matters. Anyone who tells you that a sufficiently clever prompt removes the need for human judgment is selling the "magic words" theory in a more expensive suit. The correct posture is neither credulity nor cynicism: *specify well so that checking is cheap, then actually check.*

Vocabulary

- **Prompt as specification** — treating a prompt as a complete description of a desired result (goal, inputs, constraints, output shape, quality bar, failure handling) rather than a phrase to be tuned by trial and error.
- **Output schema** — the required *structure* of the answer, independent of content: sections, named table columns, or fields. Constrains the model and makes the result checkable.
- **Acceptance criteria** — the conditions an output must satisfy to count as done, stated before the output is produced. The named test that separates "good enough" from a mood.
- **Failure handling / failure mode** — the instruction for what the model should do when it cannot meet the spec (missing, ambiguous, or contradictory input), converting silent confident guessing into a visible, actionable gap.
- **Example (few-shot)** — a concrete input-and-desired-output pair included in the prompt to demonstrate the target pattern; often more precise than a description of it.
- **Large language model (LLM)** — a system that produces likely continuations of text based on patterns learned from a large corpus; a generation engine, not a mind or a database.
- **Retrieval-augmented generation (RAG)** — pulling relevant material from a store and adding it to the prompt as inputs before the model answers, so the model works from selected context rather than general memory.
- **Semantic similarity / vector distance** — measuring relevance by closeness of meaning between numeric representations of texts, not by matching words. Captures similarity, not truth.
- **Evaluations** — systematic tests of whether a prompt or system meets its acceptance criteria across many cases, not just the one you happened to inspect.
- **Approval gate** — a required human decision before a consequential automated action executes; failure handling promoted to the workflow level.

Reading guide

OpenAI — Prompt engineering guide. <https://platform.openai.com/docs/guides/prompt-engineering>

One primary source, read deliberately, is worth more than a syllabus skimmed. Read this guide once through, then a second time against the six-part checklist in this paper. Three things to look for specifically:

1. **Its tactics are spec-parts in disguise.** "Write clear instructions," "provide reference text," "specify the desired output format" — map each tactic it lists onto Goal / Inputs / Constraints / Schema / Acceptance / Failure as you read. The guide gives you the *moves*; this paper gives you the *frame* that organizes them. Reading them together makes both stick.
2. **The reasoning behind examples.** Note where it recommends showing the model examples and *why*. Connect that to Section 3 (the model completes patterns) so the tactic becomes a principle you can apply to cases the guide never mentions.
3. **What it is careful about.** Watch for where the guide hedges — where it tells you to verify, to split complex tasks, to not over-trust a single call. Those hedges are the limitation of Section 10, stated by practitioners.

Skip, for now, anything provider-specific about a particular API's parameters. That is real but it is plumbing; the transferable skill is the specification stance, which outlives any one vendor's interface.

Walking questions

Carry these on the next walk. They are for thinking, not for answering quickly.

1. Which of the six spec-parts do you already write by instinct, and which do you routinely skip? The skipped one is where your outputs are quietly failing.
2. Take the last prompt you wrote that disappointed you. Which missing part explains the disappointment — and would adding it have fixed it, or revealed that you hadn't decided what you wanted?
3. For a task you care about, can you state the acceptance criteria out loud in one sentence? If not, the problem is upstream of the prompt.
4. Where in a knowledge-base product does "the corpus doesn't say" need to be a valid, visible answer — and what breaks if it isn't?
5. Name one bounded, low-stakes workflow you could specify fully — goal through failure handling — and test this week without risk if the model gets it wrong.

Takeaway

A prompt is not an incantation to be tuned until it works; it is a specification — goal, inputs, constraints, output shape, quality bar, and failure handling — and the output you get is bounded by how completely you write it.